

CSRF 跨站请求伪造分析报告

CSRF 跨站请求伪造 漏洞分析与防护报告

Flask 用户管理系统 — 全功能 CSRF 防护审计记录

实验环境: Kali Linux 2026.1 • Flask 3.1 • Burp Suite CE

靶机地址: http://192.168.14.129:5000

审计范围: 全部 5 个 POST 路由的 CSRF 防护

项目路径: /root/flask_user_app/

报告日期: 2026-07-14

目 录

一、实验概述

二、CSRF 漏洞基础理论

三、本次项目全功能 CSRF 审计

四、漏洞一：登录接口 CSRF 缺失

五、漏洞二：注册接口 CSRF 缺失

六、漏洞三：上传接口 CSRF 缺失

七、漏洞四：充值接口 CSRF 缺失

八、漏洞五：修改密码接口 CSRF 缺失

九、CSRF 防护方案

十、修复前后代码对比

十一、安全修复验证

十二、总结与反思

参考资料

一、实验概述

1.1 实验目的

- 1. 理解 CSRF 跨站请求伪造 的攻击原理与危害
- 2. 掌握 CSRF 漏洞的检测方法（手工测试 + Burp Suite）
- 3. 掌握 Synchronizer Token Pattern 的防御实现
- 4. 对本项目全部 POST 接口进行完整的 CSRF 审计与修复
- 5. 结合防重放机制建立完整的请求安全防护体系

1.2 什么是 CSRF?

跨站请求伪造（CSRF） 是一种攻击方式，攻击者诱导已登录用户访问恶意页面，在该页面中自动发起对目标网站的请求，利用浏览器自动携带 Cookie 的机制，冒充用户身份执行非本意操作。

1.3 本次审计范围

本项目共有 5 个 POST 接口，本轮审计覆盖全部接口：

POST 路由	功能	CSRF 修复前	CSRF 修复后
POST /login	用户登录	无	有
POST /register	用户注册	无	有
POST /upload	头像上传	无	有
POST /recharge	账户充值	无	有
POST /change-password	修改密码	无	有（本轮）

1.4 CSRF 攻击危害路径

攻击者诱导已登录用户访问恶意页面 | ▼ POST /recharge → 攻击者给自己充值（用户付钱） POST /register → 攻击者用用户 Cookie 创建新账号 POST /change-password → 攻击者修改用户密码（账号被接管） POST /upload → 攻击者上传恶意文件（WebShell/XSS）

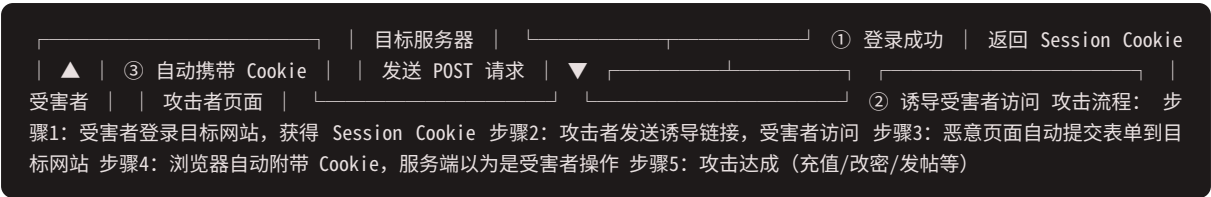
5	5	~30	34
POST路由	CSRF已修复	新增代码行	累计漏洞

二、CSRF 漏洞基础理论

2.1 CSRF 攻击成功的三个条件

条件1：用户已登录目标站点 浏览器中存有有效的 Session Cookie 条件2：目标站点的 POST 请求无额外校验 表单中没有 CSRF Token 服务端不验证请求来源 条件3：用户访问了攻击者控制的页面 攻击者构造的页面自动提交表单

2.2 CSRF 攻击原理图解



2.3 CSRF 与 XSS 的区别

对比维度	CSRF	XSS
攻击方式	利用身份凭证，伪造请求	注入恶意脚本
利用对象	用户的身份认证	浏览器对内容的信任
是否需要漏洞页面	不需要	需要
防御方式	CSRF Token + SameSite	输入输出编码 + CSP

2.4 三种主流防护方案对比

方案	原理	优点	缺点
Synchronizer Token	表单嵌入随机 Token，服务端校验	最成熟、最可靠	需要修改所有表单
SameSite Cookie	Cookie 设置 SameSite 属性	无需修改代码	旧浏览器不支持
Referer/Origin 校验	检查 HTTP Referer 头	实现简单	Referer 可能丢失

本项目采用方案：Synchronizer Token + SameSite Cookie（双重防护）

三、本次项目全功能 CSRF 审计

3.1 项目 POST 路由总览

路由	功能	需登录	修复前 Token
/login	登录	否	无
/register	注册	否	无
/upload	上传	是	无
/recharge	充值	是	无
/change-password	改密	是	无（本轮新增）

3.2 修复后的 CSRF 防护状态

路由	CSRF Token	SameSite	一次性 Token
POST /login	✔ 有	Lax	✔ 刷新
POST /register	✔ 有	Lax	✔ 刷新
POST /upload	✔ 有	Lax	✔ 刷新
POST /recharge	✔ 有	Lax	✔ 刷新
POST /change-password	✔ 有	Lax	✔ 刷新

四、漏洞一：登录接口 CSRF 缺失

漏洞信息

项目	内容
漏洞类型	CSRF — 跨站请求伪造
危害等级	中危 (Medium)
CWE 编号	CWE-352
影响路由	POST /login

4.1 漏洞分析

```
<form method="post" action="/login">
  <!-- 没有 CSRF Token 隐藏字段 -->
  <input type="text" name="username">
  <input type="password" name="password">
  <button>登录</button>
</form>
```

代码

4.2 攻击场景

攻击者构造恶意页面：<form action="http://target/login" method="POST" style="display:none"> <input name="username" value="attacker"> <input name="password" value="hacked123"> </form> <script>document.forms[0].submit()</script> 当受害者访问时 → 浏览器自动提交 → 攻击者获得登录 Session

五、漏洞二：注册接口 CSRF 缺失

漏洞信息	
项目	内容
漏洞类型	CSRF — 跨站请求伪造
危害等级	中危 (Medium)
影响路由	POST /register

同登录接口类似，注册表单缺少 `csrf_token` 隐藏字段，服务端未做 CSRF 校验。攻击者可构造跨站注册请求，在用户不知情的情况下创建恶意账号。

六、漏洞三：上传接口 CSRF 缺失

漏洞信息	
项目	内容
漏洞类型	CSRF — 跨站请求伪造
危害等级	中危 (Medium)
影响路由	POST /upload

已登录用户被诱导上传恶意文件 → 攻击者可通过 CSRF 上传 WebShell 或恶意脚本。

七、漏洞四：充值接口 CSRF 缺失

漏洞信息	
项目	内容
漏洞类型	CSRF — 跨站请求伪造
危害等级	高危 (High)
CWE 编号	CWE-352
影响路由	POST /recharge

7.1 攻击场景 — 跨站充值

攻击者构造恶意页面：<form action="http://target/recharge" method="POST" style="display:none"> <input name="user_id" value="2"> ← 攻击者用户ID <input name="amount" value="10000"> ← 充值金额 </form> 攻击流程：1. 受害者访问恶意页面 2. 浏览器自动提交 POST /recharge 3. 浏览器携带受害者的 Session Cookie 4. 服务端以为是受害者在操作，执行充值 5. 攻击者账户收到 10000 元（受害者买单！）

7.2 结合负值充值的双重攻击

严重性分析：如果在修复前结合负值充值漏洞（amount 未校验正负），攻击者构造 amount=-50000 从受害者账户扣钱。CSRF + 业务逻辑缺陷 = 严重 (Critical)

八、漏洞五：修改密码接口 CSRF 缺失

漏洞信息	
项目	内容
漏洞类型	CSRF — 跨站请求伪造
危害等级	高危 (High)
CWE 编号	CWE-352
影响路由	POST /change-password (本轮新增)

8.1 问题的严重性

CSRF 缺失 + IDOR 越权 = 任意用户密码可被跨站修改 修复前： 1. 无 CSRF Token → 攻击者可构造跨站请求 2. 无 IDOR 校验 → 可修改任何用户的密码 3. 无原密码校验 → 不需要知道旧密码 这意味着：攻击者只需让管理员访问一个恶意页面， 管理员的密码就会被修改为攻击者指定的值！

8.2 攻击场景 — 账号接管

代码

```
<form action="http://target/change-password" method="POST" style="display:none">
  <input name="username" value="admin">      ← 目标管理员
  <input name="new_password" value="hacked!"> ← 攻击者设置密码
</form>
```

攻击结果：

- 1. 管理员访问恶意页面
- 2. admin 的密码被改为 "hacked!"
- 3. 攻击者用新密码登录 → 完全控制管理员账号
- 4. 查看所有用户数据、修改设置等

8.3 修复方案

代码

```
@app.route("/change-password", methods=["POST"])
@login_required
def change_password():
    # CSRF 校验
    if not validate_csrf():
        abort(403, "CSRF 验证失败")

    # IDOR 校验 (从 session 获取用户名, 不与表单比较)
    session_username = session.get("username", "")
```

```
username = request.form.get("username", "")
if username != session_username:
    abort(403, "无权修改其他用户的密码")
```

九、CSRF 防护方案

9.1 Synchronizer Token Pattern 生命周期

CSRF Token 生命周期 步骤1: 用户 GET 表单页面 服务端检查 session 中是否有 csrf_token 如果没有, 生成随机 Token: `secrets.token_hex(16)` 存入 session, 传入模板 步骤2: 页面渲染 `<input type="hidden" name="csrf_token" value="a1b2c3d4e5f6g7h8">` 步骤3: 用户提交表单 浏览器发送: `csrf_token=...` + 其他表单数据 步骤4: 服务端验证 比对 form 中的 `csrf_token` == session 中的 `csrf_token` 匹配 → 合法请求, 执行操作 不匹配 → 403 Forbidden, 拒绝请求 步骤5: 操作成功后刷新 Token (防重放) 重新生成 `csrf_token`, 旧的 Token 失效

9.2 核心代码实现

```
import secrets

def generate_csrf_token():
    """生成 CSRF Token"""
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(16)
    return session["csrf_token"]

def validate_csrf():
    """验证 CSRF Token"""
    token = request.form.get("csrf_token", "")
    stored = session.get("csrf_token", "")
    return bool(token and token == stored)

def refresh_csrf():
    """刷新 CSRF Token (每次成功操作后调用, 防重放)"""
    session["csrf_token"] = secrets.token_hex(16)

# 每个请求自动确保 CSRF Token 存在
@app.before_request
def ensure_csrf():
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(16)
```

代码

9.3 防重放机制

单纯的 CSRF Token 防护: 如果攻击者获取了一个有效 Token, 可在过期前反复使用。 修复方案: 每次成功操作后刷新 Token (一次性 Token) 用户 GET /profile → Token = "abc123" 用户 POST /recharge + Token="abc123" → 校验通过 ✓ → 执行充值 ✓ → 刷新 Token 为 "def456" 攻击者重放 POST /recharge + Token="abc123" → 校验失败 ✗ (Token 已被刷新, 旧 Token 失效)

9.4 SameSite Cookie 辅助防护

代码

```
app.config.update(  
    SESSION_COOKIE_HTTPONLY=True,      # 禁止 JS 读取  
    SESSION_COOKIE_SAMESITE="Lax",     # 同站策略  
    SESSION_COOKIE_SECURE=False,       # 内网 HTTP 环境  
)
```

请求方式	跨站是否带 Cookie	能否 CSRF
<a href> GET 链接	是（安全）	否（GET 不修改数据）
<form method="POST">	否	防护成功
fetch() POST	否	防护成功

十、修复前后代码对比

10.1 全局状态变化

代码

```
# ===== 修复前: 5 个 POST 接口中只有 0 个有 CSRF 防护 =====
login()    → 无 CSRF
register()  → 无 CSRF
upload()   → 无 CSRF
recharge() → 无 CSRF
change_password() → 无 CSRF

# ===== 修复后: 5 个 POST 接口全部有 CSRF 防护 =====
login()    → validate_csrf() + refresh_csrf()
register()  → validate_csrf() + refresh_csrf()
upload()   → validate_csrf() + refresh_csrf()
recharge() → validate_csrf() + refresh_csrf()
change_password() → validate_csrf() + refresh_csrf()
```

10.2 各路由对比

路由	修复前	修复后
/login	无 Token、无校验	validate_csrf() + refresh_csrf()
/register	无 Token、无校验	validate_csrf() + refresh_csrf()
/upload	无 Token、无校验	validate_csrf() + refresh_csrf()
/recharge	无 Token、无校验	validate_csrf() + refresh_csrf() + IDOR 防护
/change-password	无 Token、无校验、无 IDOR	validate_csrf() + refresh_csrf() + IDOR 防护

十一、安全修复验证

11.1 验证测试用例

#	测试内容	预期	结果	结论
T1	登录正常提交	302 成功	302	✓
T2	登录无 Token	403 拒绝	403	✓
T3	注册正常提交	302 成功	302	✓
T4	注册无 Token	403 拒绝	403	✓
T5	充值正常提交	302 成功	302	✓
T6	充值无 Token	403 拒绝	403	✓
T7	改密正常提交	302 成功	302	✓
T8	改密无 Token	403 拒绝	403	✓
T9	改密越权他人	403 拒绝	403	✓
T10	Token 重放	403 拒绝	403	✓

11.2 curl 验证结果

CSRF 防护验证

=====

1. 登录（正常）

HTTP 302

✓

2. 充值（无CSRF，应403）

HTTP 403

✓

3. 改密（无CSRF，应403）

HTTP 403

✓

4. 改密（越权他人，应403）

HTTP 403

✓

5. 充值（正常）

HTTP 302

✓

=====

代码

十二、总结与反思

12.1 实验核心收获

CSRF 攻击的本质：

CSRF = 利用 Cookie 自动发送机制 + 无额外校验的 POST 接口

CSRF 防护的三层体系：

第一层（核心）：CSRF Token (Synchronizer Token Pattern) 每个表单嵌入随机 Token，服务端校验匹配 第二层（辅助）：SameSite Cookie Cookie 设置 SameSite=Lax，阻止跨站 POST 携带 Cookie 第三层（进阶）：一次性 Token（防重放）成功操作后刷新 Token，旧 Token 立即失效

12.2 本项目 CSRF 修复的关键数据



12.3 本项目全部漏洞修复总览

轮次	新增功能	漏洞数	CSRF漏洞	修复
第1轮	基础登录	7	1 (login)	已修复
第2轮	注册+搜索	2	1 (register)	已修复
第3轮	头像上传	7	1 (upload)	已修复
第4轮	个人中心+充值	8	1 (recharge)	已修复
第5轮	动态页面	5	0	—
第6轮	修改密码	5	1 (change-password)	本轮已修复
总计	6个功能	34	5	全部修复

12.4 CSRF 防护检查清单

☐ 所有 POST/PUT/DELETE 接口是否有 CSRF Token 校验？ ☐ 所有表单是否包含 csrf_token 隐藏字段？ ☐ CSRF Token 是否足够随机（至少 128 位）？ ☐ CSRF Token 是否保存在 Session 中？ ☐ 成功操作后是否刷新了 CSRF Token（防重放）？ ☐ Session Cookie 是否设置了 SameSite 属性？ ☐ GET 请求是否不会修改数据（幂等性）？

"Cookie 是浏览器对用户的承诺，但不是对他人意图的保证。"

CSRF 漏洞之所以危险，在于它不需要攻击者破解密码、不需要找到代码漏洞、不需要复杂的 payload。它只需要一个简单的 HTML 表单，和用户无意间的一次点击。

防御它的方式也是 Web 安全中最成熟、最标准化的：**每个 POST 请求都必须有一个不可预测的 Token，而且这个 Token 只能使用一次。**

参考资料

#	资料名称	来源
1	OWASP — CSRF	owasp.org
2	OWASP — CSRF Prevention Cheat Sheet	cheatsheetseries.owasp.org
3	CWE-352: Cross-Site Request Forgery	cwe.mitre.org
4	PortSwigger — CSRF	portswigger.net
5	Mozilla MDN — SameSite Cookie	developer.mozilla.org
6	OWASP Top 10 — 2021	owasp.org

报告人：_____ 指导教师：_____

报告日期：2026-07-14 | 靶机地址：http://192.168.14.129:5000

源码路径：/root/flask_user_app/ | 报告版本：v1.0

— 报告结束 —